

Trekking Management Application - V2

Overview

Adventure organizations require efficient systems to manage trekking activities involving trek organizers, staff, and participants. The Trekking Management Application (TMA) is a web application that allows Admin, Trek Staff, and Users (Trekks) to interact with the system according to their roles.

Frameworks to be Used

- Flask for API
- VueJS for UI
- VueJS Advanced with CLI, only if required
- Jinja2 templates only for the entry point when using CDN; not for UI
- Bootstrap for HTML generation and styling; no other CSS framework
- SQLite for the database; no other database
- Redis for caching
- Redis and Celery for batch jobs

All demos should be possible on the local machine. Other frameworks/libraries are not allowed apart from those specified. The database must be created programmatically through table creation or model code; manual database creation using tools such as DB Browser for SQLite is not allowed.

Roles & Functionalities

Admin

- Create, update and remove trekking routes
- Add and manage trek staff
- Assign staff to treks
- View and manage all users, staff, and treks
- Search users, staff, or treks
- Deactivate or blacklist users or staff
- View reports and trekking statistics

Trek Staff

- Log in only after being created by Admin
- View treks assigned by Admin
- Manage available slots
- Manage trek status: Open / Closed
- View registered users for assigned treks
- Update trek completion status

User (Trekker)

- Register, log in, and update profile
- View approved/open treks
- Search and filter treks by difficulty, location, and duration

- Book treks
- View booking status and trekking history

Key Terminologies

Admin

A user with the highest level of access who manages the entire trekking system.

Trek Staff

A staff member responsible for managing and coordinating assigned treks.

User (Trekker)

A participant who books and participates in trekking activities.

Trek

A trekking event created and managed in the system.

- Trek ID
- Trek Name
- Location
- Difficulty: Easy / Moderate / Hard
- Duration in days
- Available Slots
- Assigned Staff ID
- Status: Pending / Approved / Open / Closed / Completed
- Start Date
- End Date
- Extra fields as required

Booking

A record of a user booking a trek.

- Booking ID
- User ID
- Trek ID
- Booking Date
- Status: Booked / Cancelled / Completed
- Payment Status (optional)
- Extra fields as required

Staff Profile

Details of a registered staff member.

- Staff ID
- Name
- Contact Details
- Assigned Trek(s)

- Status
- Extra fields as required

The listed tables and fields are not exhaustive. Students may add or remove tables and fields as required.

Similar Applications

- AllTrails
- Thrillophilia
- Meetup

Application Wireframe

The provided wireframe illustrates the application's flow and navigation. Exact UI replication is not mandatory. Students are encouraged to design their own UI while maintaining the intended functionality and flow.

The published document references a separate file named `mad2-trekking-wireframe.pdf`.

Core Features

Authentication

- Login/register for Users (Trekksers)
- Login only for Admin and Trek Staff; no registration
- Exactly one Admin must exist and must be created programmatically
- Role-based authentication and access control using Flask security, session/token authentication, or JWT
- Unified user model differentiating user roles

Admin Functionalities

- Dashboard showing total treks, total users/staff, and total bookings
- Admin is pre-existing and created programmatically after database creation; no Admin registration
- Create and manage trek routes
- Add new trek staff
- Assign staff to treks
- View all bookings
- Search users, staff, and treks
- Deactivate or blacklist users/staff

Trek Staff Functionalities

- Dashboard displaying assigned treks and number of registered users per trek
- Update trek slots and status
- View and manage participant list
- Mark trek as started/completed

User Functionalities

- Self-register and log in
- Dashboard displaying available treks, booked treks, and trek status
- Book treks

- View booking status
- View trekking history
- Edit profile
- Search treks

Backend Jobs

Scheduled Job — Daily Reminders

- Send daily reminders to users about upcoming treks
- Include trek start date, instructions, and related information
- Delivery can use email, SMS, or Google Chat webhooks
- Runs daily at a chosen time

Scheduled Job — Monthly Activity Report

- Generate a monthly trekking activity report for Admin
- Include number of treks conducted, number of users participated, and popular treks
- Generate on the first day of every month
- Send to Admin via email as HTML or PDF

User-Triggered Async Job — Export Booking History as CSV

- Users can export their trekking history
- CSV fields: User ID, Trek Name, Location, Booking Status, and Dates
- Triggered from the user dashboard
- Export should run as a batch job and send an alert once completed

Performance and Caching

- Use Redis caching for frequently accessed treks
- Implement cache expiry
- Optimize API response times

Other Core Functionalities

- Prevent overbooking beyond available slots
- Ensure only assigned staff can manage a trek
- Allow booking only when trek status is Open
- Prevent duplicate bookings for the same trek
- Maintain complete trekking history per user
- Allow searching and filtering of treks
- Allow Admin to view all historical trekking data

Recommended / Optional Functionalities

- Well-designed PDF reports for monthly activity reports
- Charts for analytics, such as popular treks and user participation, using ChartJS
- Single responsive UI for mobile and desktop

- Unified UI across devices
- Add-to-desktop feature
- Frontend validation using HTML5 validation or JavaScript
- Backend validation within APIs
- Responsive frontend using Bootstrap
- Notifications for booking confirmation/cancellation
- Basic payment simulation
- Additional features relevant to trekking systems

Evaluation

- Submit a project report of no more than 5 pages along with the actual project submission.
- Report must include student details.
- Include project details, including the question statement and approach.
- Include an AI/LLM Declaration.
- Include frameworks and libraries used.
- Include an ER diagram with all database tables and relationships.
- Include API resource endpoints, if any.
- Include a Drive link to the presentation video.
- If AI/LLM/coding agents are used, disclose the extent of their use. Students are expected to understand the submitted code.
- A project report template is referenced in the source document.
- All code must be submitted through the portal as a single ZIP file.

Video Presentation Guidelines

- Short introduction: no more than 30 seconds
- Explain approach to the problem statement: about 30 seconds
- Highlight key application features: about 90 seconds
- Discuss additional features: about 30 seconds
- Final video should not exceed 5–10 minutes.
- Keeping the video feed on is optional but recommended.
- Upload the video to the student Drive with access for anyone with the link and include the link in the report.

Viva

- After the video explanation, demonstrate the application.
- Answer examiner questions.
- Be prepared to make requested changes and run the code during the live demo.
- Questions may also cover course-relevant topics unrelated to the project itself.

Instructions

- The document is live and may be updated with more details, including the wireframe.

- The problem statement is intended to be frozen on or before the date specified by the course team; subsequent modifications will be communicated through official announcements.
- The project must be submitted as a single ZIP file.